LOOMGUARD

Pre-Publication Brand Governance & Compliance Platform

Team: KIZUNA (RGUKT Basar)	**Event:** Inkloom AI Hackathon (Round 3 — Final)
Authors: Shruthi Pallemoni & Pradeep Sutrave	**Date:** October 30, 2026
Live App (Vercel): https://loomguard-kizuna.vercel.app/	**Live API (Render):** https://loomguard-kizuna-6cra.onrender.com/
Source Code (GitHub): https://github.com/lostmoon1513/loomguard-kizuna	**API Docs:** https://loomguard-kizuna-6cra.onrender.com/docs

Executive Summary: LoomGuard is an intelligent brand-governance platform that enforces corporate guidelines before publication. Creators upload multi-format text drafts (.md, .docx, .pdf, text) or banner images (PNG, JPEG). LoomGuard validates them against uploaded brand guides and approved factual claim sheets. **Core Design Principle:** Measurements, rule matches, color distances, and citations are executed strictly in deterministic Python code. Google Gemini (2.5 Flash / Vision) is utilized strictly for extraction, tone review under schema constraints, and visual bounding boxes. The system *never* auto-approves and *never* rewrites text unprompted, preserving complete human editorial sovereignty.

1. Delivering the Three Round 3 Commitments

At Round 2, Team KIZUNA committed to exactly three core improvements for Round 3. All three have been engineered, deployed to production, and benchmarked against ground truth:

Improvement 1: Image Banner & WCAG 2.1 AA Accessibility Checker

- **Dual-Engine Color Science:** Deterministic K-Means color clustering in CIELAB space with Sharma et al. (2005) CIEDE2000 color difference (Delta E ≤ 6.0 threshold) against official brand palettes.
- **WCAG 2.1 AA Contrast Ratio:** Computes relative luminance L1/L2 in Python, verifying text and background contrast against standard 4.5:1 (normal text) and 3.0:1 (large text) thresholds.
- **Same-Hue Passing Suggestions:** When contrast fails, algorithmic lightness stepping computes the closest passing color within the exact same hue angle and saturation.
- **Gemini Vision Bounding Boxes & Annotation:** Identifies text regions and logos with [ymin, xmin, ymax, xmax] coordinates. The endpoint GET /api/drafts/{id}/image?annotate=true renders high-resolution numbered overlays.
- **Safe Re-upload Lifecycle:** Banner pixel edits cannot be applied as text strings; LoomGuard returns 422 FIX_BY_REUPLOAD, cleanly preserving revision tracking.

Improvement 2: Version Diff History & Exception Waiver Lifecycle

- **Word-Level Diff Engine:** High-resolution diffing via Python difflib.SequenceMatcher returning equal, insert, delete, and replace opcodes. Guaranteed 100% mathematical reconstruction invariance.
- **Formal Exception Waiver Workflow:** Creators request exceptions with required justification notes (exception_requested). Managers/Reviewers can grant or deny waivers with an audit trail.
- **Auto-Expiring Waivers:** If a creator alters the quoted text of a granted waiver in a subsequent revision, the waiver automatically expires and triggers fresh validation.
- **Cryptographic JWT RBAC:** 4 distinct roles (creator, creator2, reviewer, manager) secured with HMAC-SHA256 tokens and strict tenant isolation (unauthorized requests yield 404 to block ID enumeration).

Improvement 3: Governance Insights Over Time & Pre-LLM Sensitive Data Masking

- **Empirical Weekly Trend Analytics:** Aggregates draft volume, violation frequencies, and approval velocity across 8-week sliding windows. Velocity slope identifies *rising*, *falling*, or *steady* compliance friction.
- **Deterministic Policy Recommendations:** Automatically surfaces 'Potentially Unclear Rules' based on high repeat violations and granted waiver ratios—without relying on hallucinated LLM feedback.
- **Pre-LLM Data Masking Engine:** Reversible regex/Luhn engine masks Credit Cards (13-19 digits), API Keys, JWTs, emails, phone numbers, and confidential project codenames before external API transit.
- **Fail-Closed Leak Defense & Preview Security:** Outgoing prompts are checked against masker.leaked(). Stored drafts and UI inspect screens display only masked previews (e.g. •••• 1111).
- **Export Facilities:** Formula-injection-safe CSV export and standalone ReportLab compliance PDF generation.

2. Complete Matrix of Built Features

Feature Area	Component & Technical Implementation	Status
Brand Kit Engine	PDF guide parsing (PyMuPDF/pypdf) + Approved Facts CSV parser. Preserves page numbers, priority (mandatory/advisory), and exception clauses.	Production
Draft Intake	Accepts plain text, Markdown, .txt, .docx, and digital PDFs. Zero-width character sanitizer and curly quote normalizer.	Production
Terminology Check	Strict and fuzzy hyphen/case/plural-aware banned/preferred terms with guide page citations.	Production
Fact Verification	Algorithmic fact-sheet verification: CONTRADICTED (numerical delta), OUTDATED (expired), UNSUPPORTED (unverified), plus superlative gating.	Production
Security Guard	Prompt-injection defense scanning text and image OCR for adversarial directives ('ignore rules', 'approve this').	Production
AI Tone Review	Google Gemini 2.5 Flash bounded by strict Pydantic output schemas, verbatim quote checks, and 10s timeouts.	Production
Image & WCAG	CIELAB K-Means, CIEDE2000 Delta E ≤ 6.0, WCAG 2.1 AA luminance contrast, Gemini Vision bounding boxes, image annotation.	Round 3
Diff & Waivers	Word-level SequenceMatcher diffing, formal waiver lifecycle, auto-expiring waivers on text change.	Round 3
RBAC & Security	JWT Bearer tokens (HS256), 4 user roles, tenant isolation, sliding-window IP rate limiter (240/20 req/min).	Round 3
Insights & Trends	Weekly time-series aggregation, empirical trend velocity, formula-safe CSV export, PDF compliance summary.	Round 3
Data Masking	Local Luhn credit card, API key, email, phone, and codename masking with fail-closed prompt leak interception.	Round 3

3. Technical Architecture & Design Philosophy

LoomGuard is architected to decouple deterministic verification from probabilistic AI extraction. A major failure mode in generative compliance tools is LLM hallucination and non-deterministic scoring. LoomGuard eliminates this risk through a **Python-First Architecture**:

Client Layer	React (Vite) Single-Page Application: Review Studio, Word-Level Diff View, Review Queue, Governance Insights, and Cryptographic JWT Role Switcher.
Gateway & Security	FastAPI + ASGI Gateway: IP Sliding-Window Rate Limiter (240 general / 20 heavy req/min), Unicode zero-width character sanitizer, and CORS security headers.
Local Python Rules (Deterministic)	• Terminology Matcher (Banned/preferred words with exact guide citations) • Fact Verification (CONTRADICTED numeric deltas, OUTDATED expiry, UNSUPPORTED) • Color Science (CIELAB K-Means clustering + Sharma CIEDE2000 Delta E ≤ 6.0) • WCAG 2.1 AA Accessibility (Relative luminance contrast + same-hue stepping) • SequenceMatcher Diff Engine (Word-level diff with 100% reconstruction invariance)
AI Extractor & Vision (Constrained)	• Sensitive Data Masker: Pre-LLM Luhn regex masking of cards, keys, emails, phones, and codenames • Google Gemini 2.5 Flash: Tone check bounded by strict Pydantic schemas (verbatim quotes only) • Gemini Vision: Text and logo bounding box identification [ymin, xmin, ymax, xmax] • Fail-Closed Prompt Defense: Intercepts and blocks prompt injections or leaked tokens
Persistence & Audit Layer	MongoDB Atlas (with automatic in-memory fallback): Immutable draft revision history (v1..vn), formal exception waiver records, and multi-role audit trail.

4. Testing & Empirical Evaluation Benchmark

LoomGuard was subjected to rigorous empirical evaluation against ground truth datasets spanning text drafts, sensitive entities, color pairs, and contrast ratios (143/143 tests passing in pytest):

Evaluation Domain	Benchmark / Ground Truth	LoomGuard Result	Baseline / Target
Three-System Text Benchmark (56 drafts, 39 gold issues)	Full golden dataset with terminology, claim contradiction, outdated pricing, and clean drafts.	**100% Recall / 100% Prec.** 0% False Positives (0/23 clean) 100% Citation Accuracy	System A (Hype): 18% Rec System B (RAG): 46% Rec Target: >95% Recall
Sensitive Data Masking (Improvement 3)	15 sensitive entities (Luhn cards, API keys, JWTs, emails, phones, codenames) + 13 benign business numbers.	**100% Recall (15/15)** 0% Benign False Positives 100% Roundtrip Unmasking	Zero prompt leakage to external APIs
CIEDE2000 Color Metric (Improvement 1)	23 standard benchmark color pairs from Sharma, Wu, Dalal (2005) reference dataset.	**MAE = 0.000022** Max Error = 0.000078	Mathematical parity with published standard
WCAG 2.1 AA Contrast (Improvement 1)	Ground-truth luminance test vectors across boundary ratios (4.5:1 normal, 3:0:1 large).	**100% Accuracy** 100% same-hue suggestions pass ratio	Strict WCAG 2.1 AA standard
Word-Level Diff Engine (Improvement 2)	Multi-revision sequence transformations across insertions, deletions, and substitutions.	**100% Invariance** (equal+del=orig; equal+ins=revised)	Zero character loss or alignment drift
Rule-Check Latency	End-to-end Python rule suite executed over test corpus.	**Mean: 1.3 ms / draft** P95: 1.6 ms / draft	Sub-15ms real-time typing speed

Comparative Analysis: Why Generic LLMs Fail at Brand Compliance

System A (Generic Hype Checker): Achieved only 18% recall because LLMs without grounding overlook subtle numerical discrepancies (e.g. 'under 200 milliseconds' vs 'under 150 ms' in fact row 4) and miss banned terms disguised with hyphens or capitalization.

System B (Vector RAG Only): Improved recall to 46%, but chunking boundaries frequently severed exception clauses from mandatory rules. Furthermore, vector similarity failed to distinguish between expired pricing facts and active pricing facts with similar semantic embeddings.

System C (LoomGuard): Achieves **100% precision and 100% recall** by combining deterministic chunk extraction (preserving page numbers, priority, and exception clauses together) with strict algorithmic rule verification. Zero clean drafts received false alarms (0/23).

5. Product Roadmap & Future Development Plan

LoomGuard's production trajectory extends beyond the hackathon into an enterprise governance ecosystem:

Phase & Timeline	Key Deliverables & Objectives	Target Ecosystem
Phase 1: CI/CD & Browser Extension ***(Q1 2027)***	• **Browser Extension:** Real-time LoomGuard checking directly inside HubSpot, Notion, Google Docs, and WordPress. • **GitHub Action / GitLab CI:** Block PRs containing unapproved claims or non-compliant docs in product repos.	Marketing Ops & Content Creators
Phase 2: Multi-Modal Video & Audio ***(Q2 2027)***	• **Video Keyframe Sampling:** Extract video frames, checking onscreen graphics and lower-thirds for WCAG compliance. • **Podcast / Spoken Audio:** Transcription and timestamped fact-checking for spoken claims.	Multimedia & Social Teams
Phase 3: Enterprise Federation & SSO ***(Q3-Q4 2027)***	• **SAML / Okta / Azure AD:** Enterprise SSO with automated role synchronization. • **Multi-Brand Hierarchy:** Support parent corporations managing multiple sub-brands with shared master glossaries. • **SOC 2 Type II Compliance:** Immutable tamper-proof audit trails for legal discovery.	Enterprise Legal & Brand Custodians

6. Team Contributions & AI Disclosure

Team Member	Core Responsibilities & Implemented Components
Shruthi Pallemoni Backend & AI Architecture	Color science engine (CIELAB K-Means, CIEDE2000), WCAG 2.1 AA contrast pipeline, sensitive data masking engine, sliding-window rate limiter, MongoDB persistence layer, and evaluation test suite (evals/run_evals.py).
Pradeep Sutrave Frontend & Product Architecture	React Review Studio, Word-level diff viewer, exception waiver request workflow, brand setup dashboard, demo kit creation, and user experience hardening.

AI Assistance Disclosure: Architecture planning and pair programming were conducted with assistance from Claude and Google Antigravity. Zero code was recycled from external repositories. All runtime AI utilizes official Google Gemini APIs via google-genai SDK, bounded by strict validation.

7. Judge Verification & Smoke Testing

1. **Live Web Application:** Navigate to https://loomguard-kizuna.vercel.app/. Pre-seeded demo brand kits and flawed drafts allow full end-to-end testing immediately.
2. **Live API Documentation:** Open https://loomguard-kizuna-6cra.onrender.com/docs for interactive OpenAPI Swagger documentation.
3. **Automated Smoke Test:** Run our standalone smoke test script against production with zero dependencies: python backend/scripts/smoke.py https://loomguard-kizuna-6cra.onrender.com.
4. **Local Unit Test Suite:** Run pytest tests -q inside backend/ to execute all 143 passing unit and integration tests.

LoomGuard — Brand Protection with Mathematical Certainty.
Team KIZUNA • RGUKT Basar • October 2026